

CyberAudit & Solutions

MVP technical documentation

Stage 3 — Reusable for RNCP 5
Web and Mobile Web Developer

Technical expert	Tommy Jouhans	Front-End Developer
Technical expert	James ROUSSEL	Back-End Developer



Holberton School Dijon — Class of 2026

Summary

1. Introduction

- 1.1 Context
- 1.2 Scope of the document
- 1.3 Target audience
- 1.4 technical stack

2. User Stories & Mockups

- 2.1 Personas
- 2.2 Prioritized User Stories (MoSCoW)
- 2.3 Screen descriptions (11 mockups)

3. System architecture

- 3.1 Overview — Layered Architecture
- 3.2 Data flow — Audit request
- 3.3 Justified architectural choices
- 3.4 Accessibility (WCAG 2.1 AA)

4. Components, classes and databases

- 4.1 Database Schema
- 4.2 Back-end class diagram
- 4.3 Front-end components (React + Tailwind)
- 4.4 PDF report generation
- 4.5 Monitoring & Observability

5. Sequence diagrams

- 5.1 JWT Registration + Login
- 5.2 Submitting an audit request (async)
- 5.3 Generating the PDF report (admin)

6. API documentation

- 6.1 External APIs consumed
- 6.2 Internal API — REST Endpoints

6.3 Cross-cutting conventions

7. **SCM & QA Strategy**

7.1 Source Code Management (Git)

7.2 QA Strategy

7.3 Manual safety tests

7.4 CI/CD Pipeline

7.5 GDPR Compliance

8. **Appendices**

8.1 Glossary

8.2 Mapping Stage 3 ↔ RNCP 5

8.3 Sources & technical references

8.4 Project risk matrix

8.5 Internal links within the project

1. Introduction

1.1 Context

CyberAudit & Solutions is a full-stack web platform that connects French SMEs/VSEs without an internal IT team with cybersecurity experts. According to ANSSI 2023, 45% of SMEs have no dedicated IT team and remain exposed to cyberattacks (ransomware, phishing, data breaches).

1.2 Scope of the document

This technical documentation covers the detailed design of the MVP. It describes:

- User needs formalized into prioritized User Stories
- Software architecture and data flow
- The data model (relational PostgreSQL)
- Back-end classes, front-end components, interaction scenarios
- The APIs exposed and consumed
- The code management and quality strategy

1.3 Target audience

- Development team (Tommy + James) — reference during the coding phase (W2 → W12)
- Holberton Jury — proof of the MVP's detailed design
- RNCP Jury 5 — reusable skills portfolio for Web and Mobile Web Developer certification

1.4 Technical Stack

Layer	Technology	Justification
Front-End	React 18 + Tailwind CSS	High-performance SPA, rapid design system
Back-End	Django 5 + DRF	Native security, mature ecosystem
BDD	PostgreSQL 15	ACID relational analysis, high-performance indexes
Async	Celery + Redis	Long-running tasks outside of HTTP threads
Auth	SimpleJWT	Stateless, scalable
Accommodation	Vercel + Railway	Automatic HTTPS, Git deployment

2. User Stories & Mockups

2.1 Personas

Married—45 years old, manager of an accounting firm in Dijon, no IT knowledge. UX target < 3 min.

Karim—32 years old, cybersecurity expert, manages the queue of requests in parallel.

2.2 Prioritized User Stories (MoSCoW)

Must Have

ID	User Story	Internship
US-01	As a visitor, I want to register in order to create my account.	W3
US-02	As a user, I want to log in to access my account.	W3
US-03	As a user, I want to log out in order to protect my account.	W3
US-04	As an SME customer, I want to submit a request in < 3 min.	W4
US-05	As a customer, I want to track the status (4 statuses).	W4
US-06	As a customer, I want to receive an email for every change.	W4
US-07	As an admin, I want to see all requests sorted.	W5
US-08	As an admin, I want to change the status.	W5
US-09	As an admin, I want to generate a PDF.	W5
US-10	As a customer, I want to download my PDF.	W5

Should Have

US-11 awareness modules • US-12 MDP reset • US-13 admin filters • US-14 UI notifications.

Could Have / Won't Have

US-15 to US-17 (V2). Outside scope: mobile app, AI score, marketplace, OAuth2, gamification.

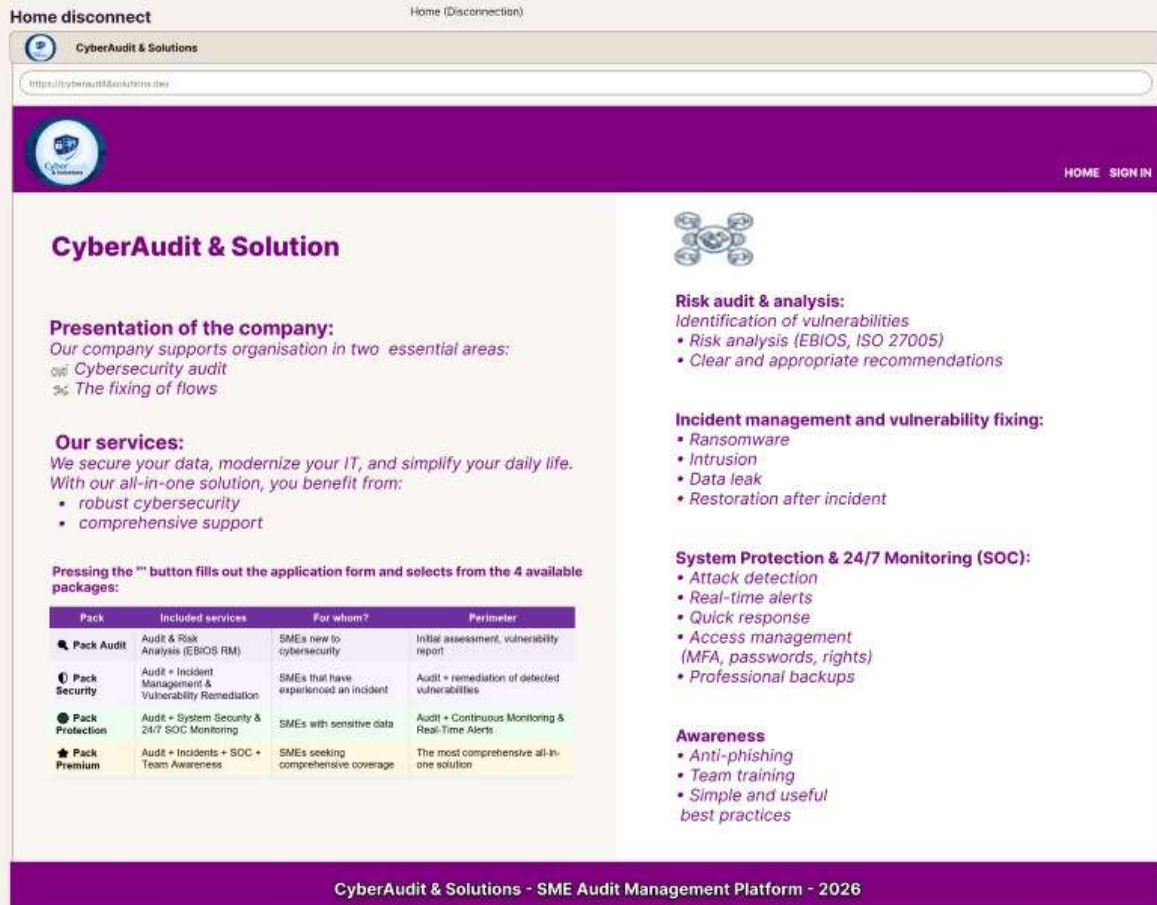
2.3 Screen Descriptions (11 Figma mockups to be produced in W1)

Screen 1 — Public Landing

Hero (ANSSI issue), CTA, benefits. The CyberAudit & Solutions platform's homepage presents the company, its services, and the packages offered, tailored to entrepreneurs with fewer than 50 employees. Simply clicking on the "SIGN IN" tab allows the client (Marie) to create an account and log in.

1. Public Access & Authentication

Screens visible before login: public home, login/registration, logged-in home



Mockup 1 — Public Landing

Screen 2 — Registration

Email + Password + Terms and Conditions/GDPR. After clicking "Create an account," the customer enters their company name, first name, last name, email address, and password (hashed). Form completion rules:

- Each field contains a maximum of 50 characters
- If a field is missing: error "please enter the information on the form"
- Email format: `domain@name` OBLIGATORY
- Password: one lowercase letter, one uppercase letter, one number, one special character, minimum 10 characters

Create an account

Log in

CyberAudit & Solutions

https://cyberauditandsolutions.com/

CyberAudit & Solutions

HOME SIGN IN

Create an account

Fill in the information below to get started

Company Name

Enter company name

First Name

Enter first name

Last Name

Enter last name

Email

Enter email address

Password

Create a password

Create an account

Already have an account?

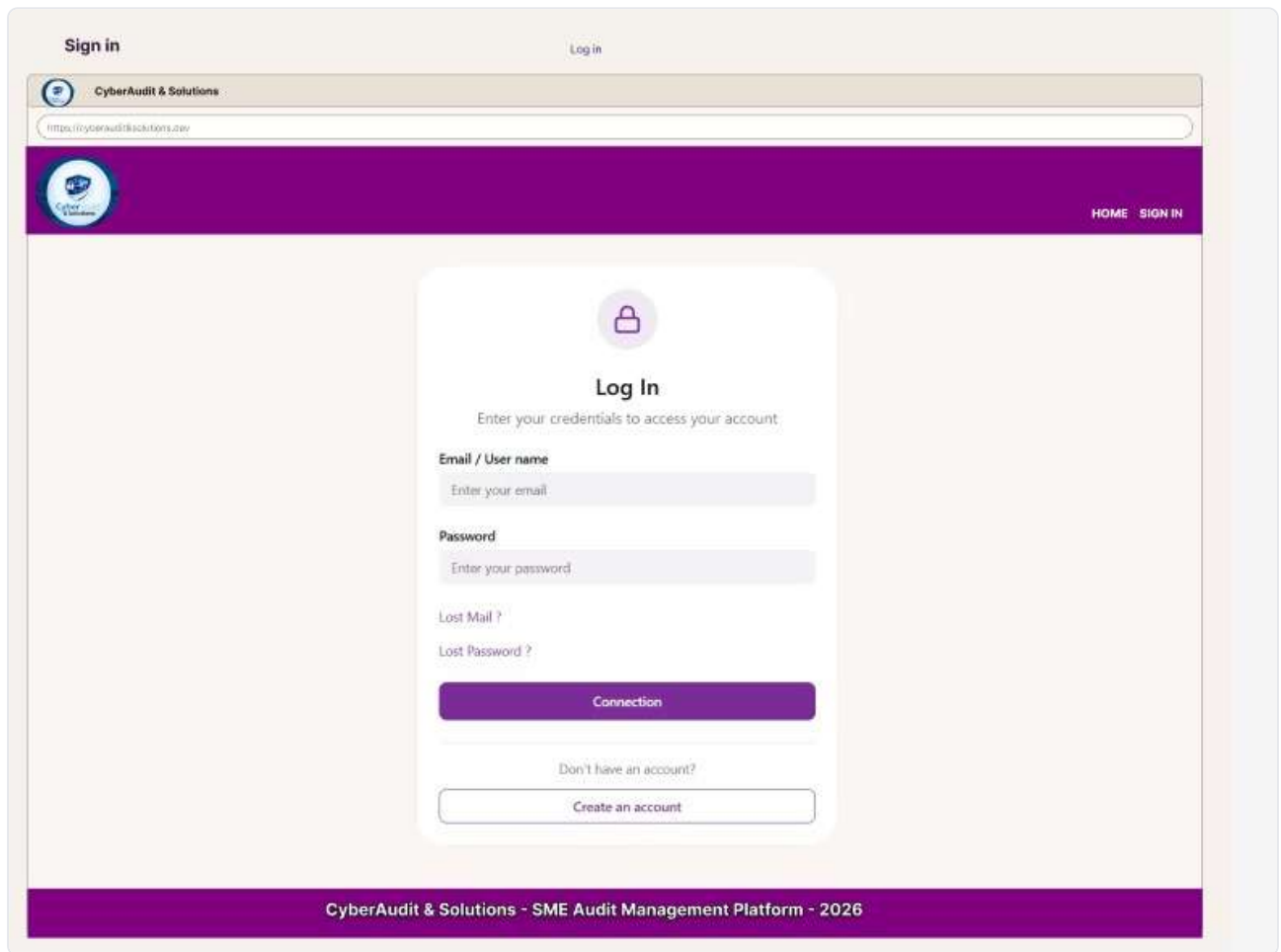
Sign In

CyberAudit & Solutions - SME Audit Management Platform - 2026

Mockup 2 — Inscription

Screen 3 — Login

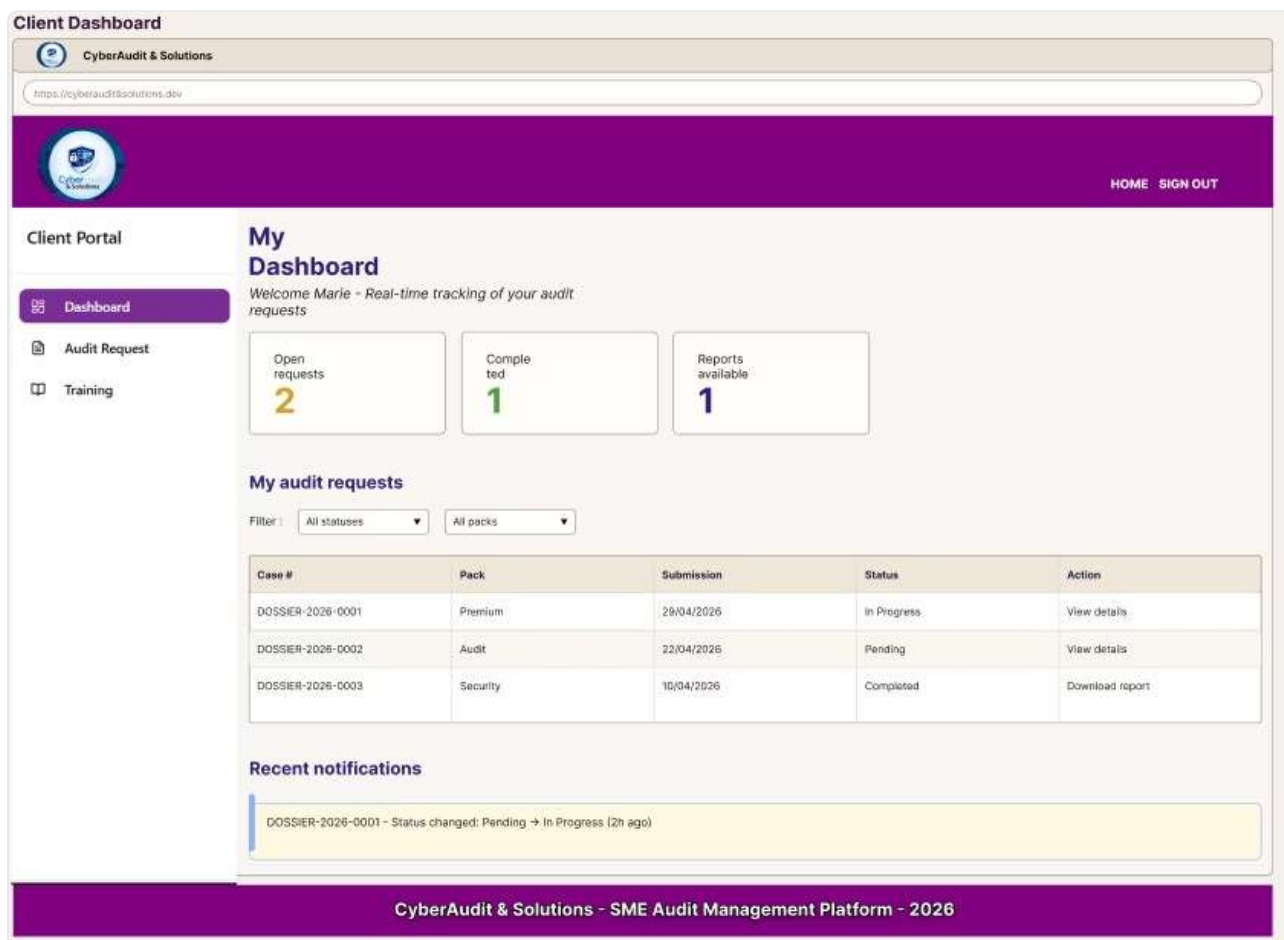
Email + Password. The client can log in with her account; otherwise, she clicks on "Create an account". "Lost Mail" and "Lost Password" links are planned (to be developed later for the RNCP 5 presentation).



Mockup 3 — Login

Screen 4 — Client Dashboard

Sidebar, request list, floating CTA. When the client is logged in, she can view her dashboard to track the progress of her audit request (Pending / In Progress / Completed). She can click on "View details" and download the report via "Download report".



Mockup 4 — Client Dashboard

Screen 5 — Audit Form

Stepper 3 steps, target < 3 min. The client enters their username, company name, and package selection (Audit / Security / Protection / Premium) with a display of "Included Services," and an optional message. A "Send the Audit Request" button is then clicked to send the request.

The image shows a web browser window displaying the 'CyberAudit & Solutions' Client Portal. The browser's address bar shows the URL 'https://cyberauditmockup10kx.dev/'. The portal has a purple header with the company logo and navigation links for 'HOME' and 'SIGN OUT'. A left sidebar contains a 'Client Portal' section with links to 'Dashboard', 'Audit Request' (highlighted), and 'Training'. The main content area is titled 'Audit request form' and contains the following fields and options:

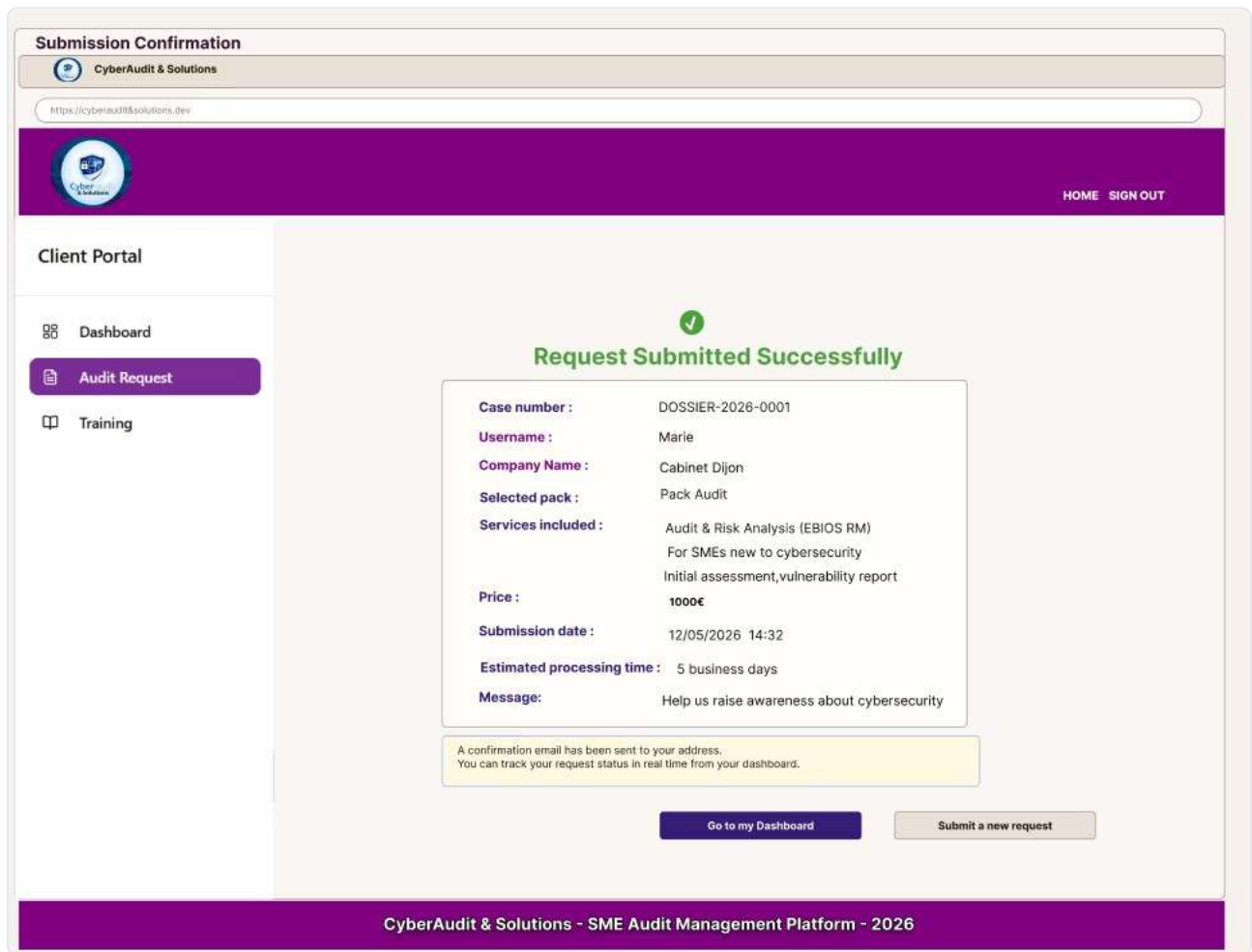
- Username :** A text input field.
- Company Name :** A text input field.
- Select pack :** A group of four radio buttons:
 - ☒ Pack Audit
 - ☐ Pack Security
 - ☐ Pack Protection
 - ☐ Pack Premium
- Included Services:** A text area containing the text: 'Print in the panel of included services of the selected package such as: If you select, "Audit & Risk" Display for example: Analysis (EBIOS RM) | SMEs new to cybersecurity | Initial assessment, vulnerability report | 5 business day | Price: 1000 €'.
- Message :** A large text input field.
- Buttons:** A purple button labeled 'Sent the audit request' is located below the message field.

The footer of the portal is purple and contains the text 'CyberAudit & Solutions - SME Audit Management Platform - 2026'.

Mockup 5 — Audit Form (3 steps)

Screen 6 — Request details (customer)

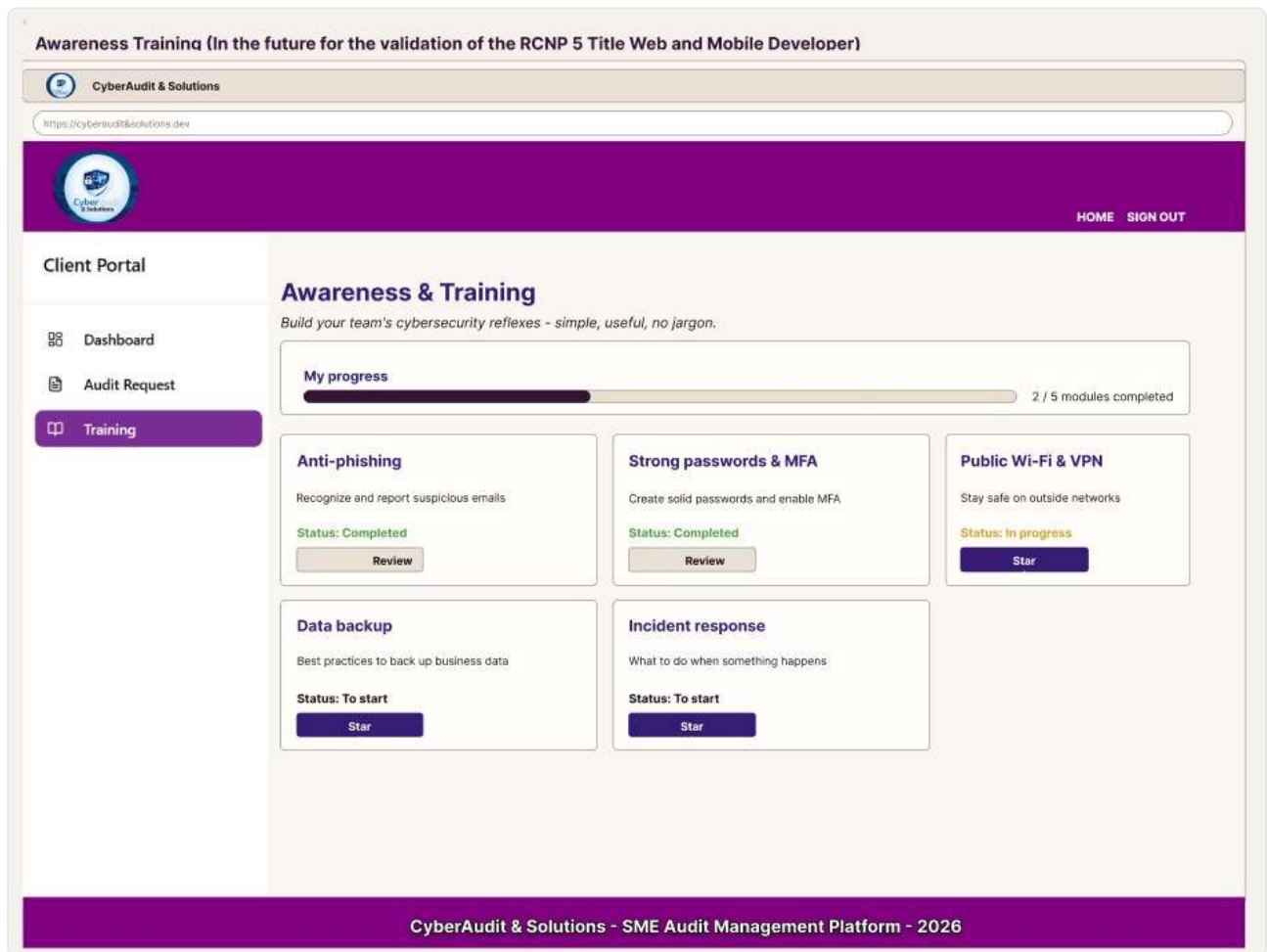
Status, timeline. After sending, a confirmation message is sent to the expert (Karim) and an acknowledgement of receipt is sent to the client. The client then tracks the progress of the analysis.



Mockup 6 — Customer-side request detail

Screen 7 — Training Module

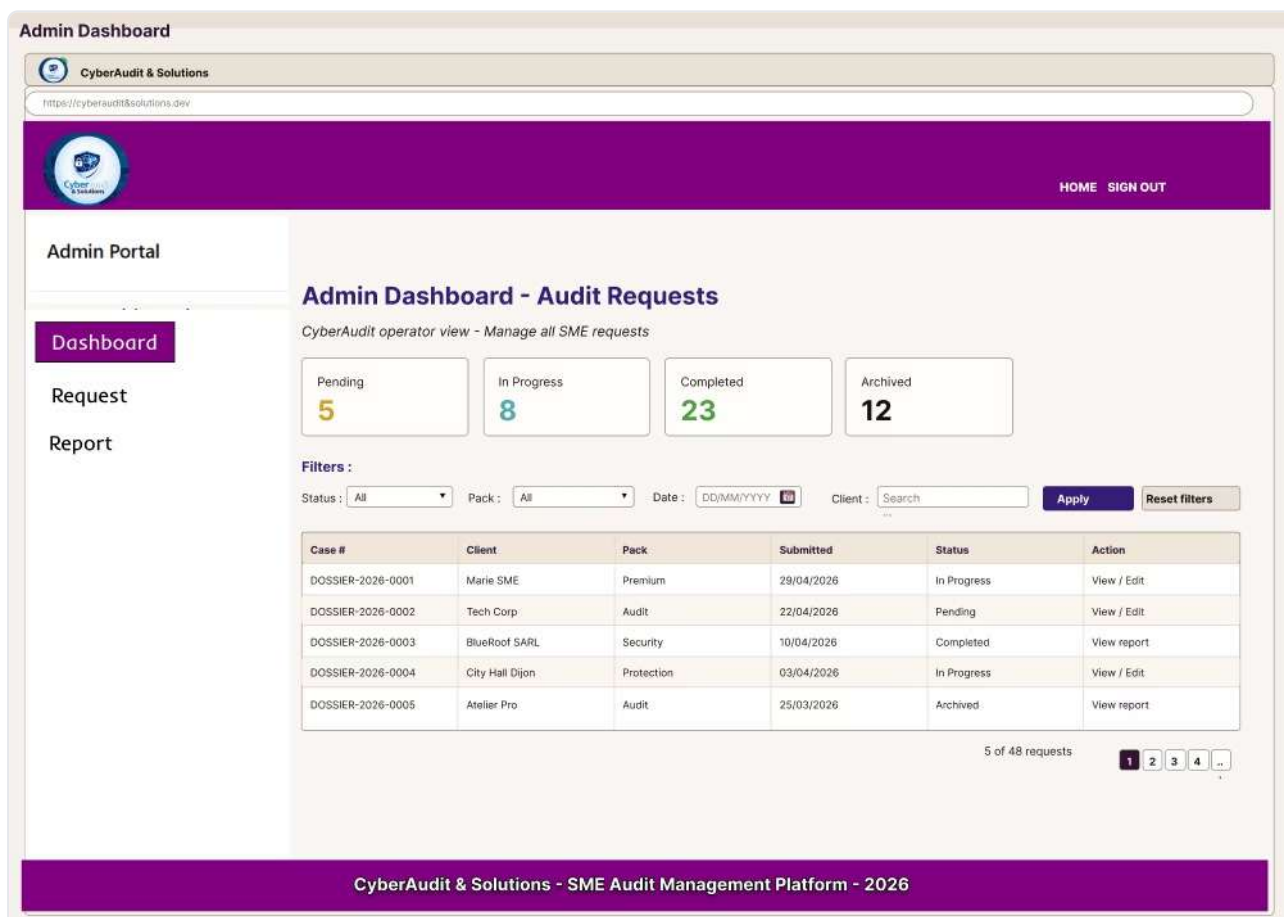
Map list + detailed Markdown. (To be developed later). Modules: "Anti-phishing", "Multi-factor authentication", "Public connection and Wi-Fi & VPN", "Data backup", "Incident response". Available in various formats (video, video conferencing, documentation) with a progress bar.



Mockup 7 — Training Modules

Screen 8 — Admin Dashboard

Table + filters. The expert (Karim) accesses his dashboard with all requests. He can sort the list according to the chosen package and current statuses, then view and edit each request.



Mockup 8 — Admin Dashboard

Screen 9 — Request details (admin)

Status selector + generate button. By clicking on "View/edit", Karim changes the status via the "Status & Actions" form, generates a PDF report via "Generate PDF report", sends a notification to the client ("Send notification") and can archive the request ("Archive request").

Admin Request Detail

CyberAudit & Solutions

https://cyberauditsolutions.dev



HOME SIGN OUT

Admin Portal

Dashboard

Request

Report

Request DOSSIER-2026-0001

Back to list

Client information

Company :

Marie SME (Cabinet Comptable)

Contact :

Marie Dupont

Email :

marie@cabinet-dijon.fr

Submitted :

29/04/2026 at 14:32

Pack selected :

Pack Audit

Client message :

Help us raise awareness about cybersecurity

Status & Actions

Current status :

In Progress

Assigned to :

Karim

Internal notes

EBIOS RM analysis ongoing. Awaiting client log files.

Save changes

Generate PDF report

Send notification

Archive request

History (immutable - GDPR)

Date	Author	Action
29/04/2026 14:32	Marie Dupont	Request submitted (Pack Premium)
29/04/2026 14:32	System	Confirmation email sent (Celery)
29/04/2026 16:10	Karim	Status changed: Pending → In Progress
29/04/2026 16:11	Karim	Internal note added

CyberAudit & Solutions - SME Audit Management Platform - 2026

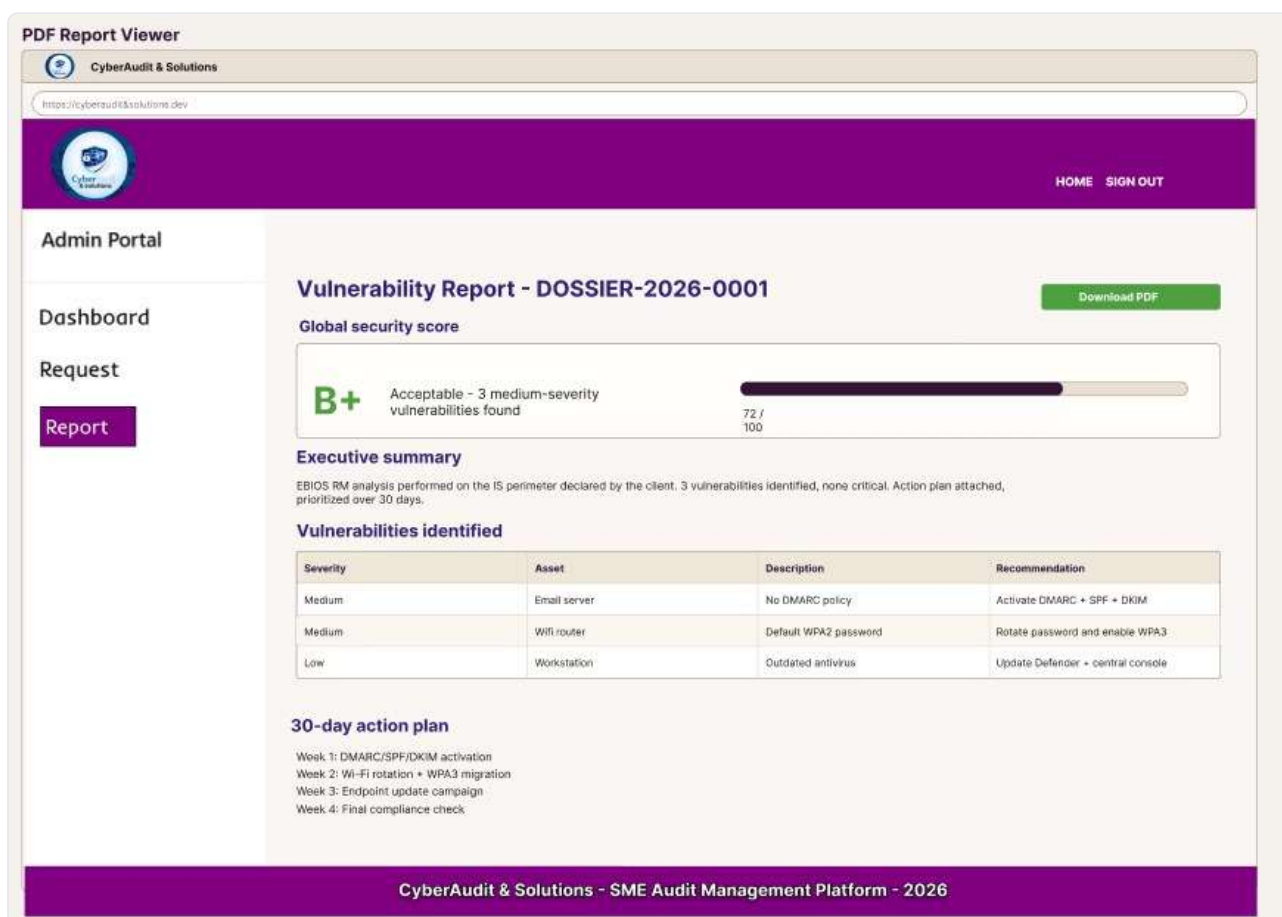
Mockup 9 — Detail request on the admin side

Screen 10 — PDF Report Viewer

The report can be downloaded. The report identifies:

- The level of vulnerability (Low, Medium, High)
- The advantages (mail server, wifi router, etc.)
- The description
- Recommendations

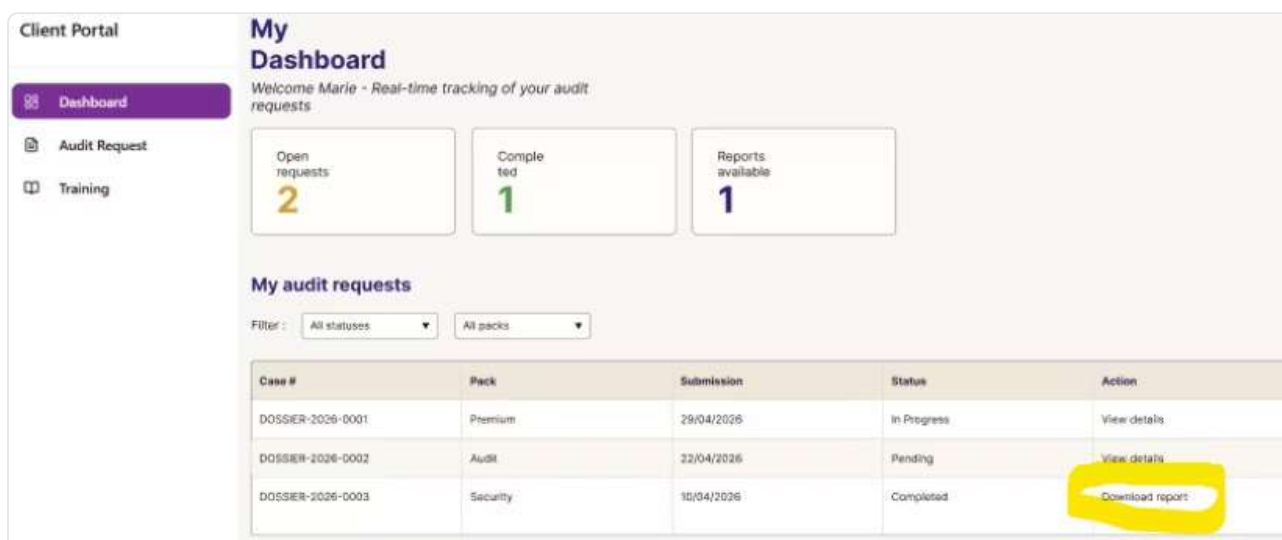
An action plan is generated with the estimated number of days.



Mockup 10 — PDF Report Viewer

Screen 11 — Download from the dashboard

The client can download the report from their dashboard via the "Download report" link.



Mockup 11 — Download from the dashboard

3. System Architecture

3.1 Overview — Layered Architecture

The architecture follows a layered model: Client (browser) → CDN/Backend → Data + External Services.

- **Client (Browser):** React SPA · Tailwind CSS · DOMPurify (anti-XSS), HTTPS, REST + JWT
- **Vercel / CDN:** Static assets · HTTPS Let's Encrypt
- **Back-End (Railway / Render):** Django REST API (JSON, JWT, HTTPS) — Celery Worker (async) — Redis (broker) — ORM — SMTP
- **Data layer:** PostgreSQL 15 + PDF Storage
- **External services:** SMTP Gmail (email notifications)

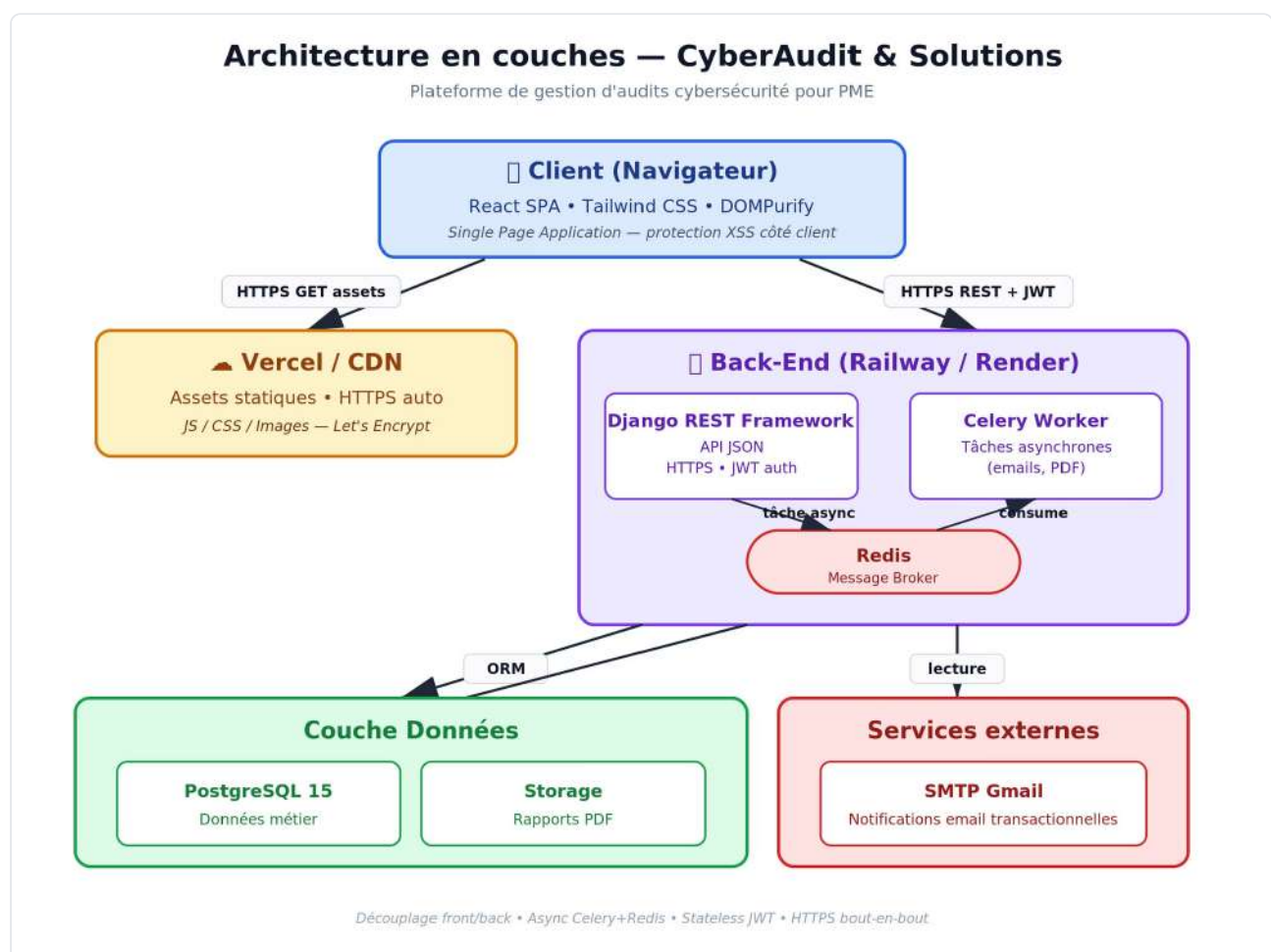


Figure 1 — Layered architecture: Client → CDN/Backend → Data + External Services

3.2 Data Flow — Audit Request

1. Marie fills out the form in the SPA React.
2. The front sends `POST /api/audits/` (JSON + JWT in `Authorization: Bearer`).
3. Django validates the request (DRF serializer) and persists in the BDD via the ORM.
4. Django publishes a Celery task `send_acknowledgment_email(audit_id)` in Redis.

5. Django responds `Created in 201` with details of the audit.
6. Karim (admin) sees the new request in his dashboard (polling 30 s — V2: WebSocket).

3.3 Justified architectural choices

- **Front/back decoupling**→ 2 independent deployments, CORS controlled, SPA cacheable on CDN.
- **Async Celery**→ Generating a PDF can take 5-10 seconds; we are not blocking the HTTP thread.
- **Relational PostgreSQL**→ strongly structured data with relationships (User → AuditRequest → Report), ACID integrity critical.
- **JWT stateless**→ no server session, scalable horizontally.

3.4 Accessibility (WCAG 2.1 level AA)

The primary target user (Marie, 45 years old, non-IT) requires an accessible user experience. The platform aims for compliance.**WCAG 2.1 Level AA**based on the 4 principles: perceptible, usable, understandable, robust.

3.4.1 Rules applied

WCAG Principle	Implementation
Minimal contrast	Ratio 4.5:1 for normal text, 3:1 for wide text. <code>Palettecyber-700 (#5b21b6)</code> on a white background validated — verification tool:DevTools axis.
Keyboard navigation	All interactive elements accessible viaTab / Shift+Tab .Logical focus order respecting the reading flow.focus-visibleTailwind with a 2px purple ring on each CTA.
ARIA labels	<code>Status Badgeexposedaria-label="Status: in progress" .</code> <code>SecurityScoreGaugeexposedrole="meter" + aria-valuenow .Decorative icons in aria-hidden="true" .</code>
Forms	Each <code><input></code> has a <code><label></code> associated (not just a placeholder). Error messages linked via <code>aria-describedby</code> .Required fields are indicated with <code>aria-required</code> .
Semantic structure	Tags <code><header></code> , <code><nav></code> , <code><main></code> , <code><footer></code> ,hierarchy of titles <code>h1→ h6</code> No skipping. Skip-link "Go to content" in the first position.
Images and icons	Attribute <code>altSystematic</code> : descriptive if informative, blank if decorative. Architectural diagrams provided in accessible SVG format with equivalent textual description.
Multilingual	Attribute <code>lang="fr"</code> on <code><html></code> .No content mixing FR/EN without change of lang.
Animations	Respect for <code>prefers-reduced-motion</code> via media query Tailwind: transitions are disabled if the user requests it in their OS.

3.4.2 Automated Accessibility Tests

- **axe-core** integrated into Vitest tests: `npm install -D @axe-core/react-jest-axe`. Each major component (LoginForm, AuditRequestForm, RequestTable) tested via `expect(await axis(container)).toHaveNoViolations()`.
- **Lighthouse CI** in the GitHub Actions pipeline: Accessibility score ≥ 90 required for merge.
- **W7-W10 Manual Tests**: 100% keyboard navigation on the 3 critical paths (registration, audit submission, PDF download), screen reader test with NVDA (Windows) and VoiceOver (macOS).

3.4.3 Measurable Target

Lighthouse Accessibility Score $\geq 90/100$ on the 4 main pages (landing, login, client dashboard, audit form) before the W12 presentation.

4. Components, Classes and Databases

4.1 Database Schema

Entity	Key fields	Relationships
USER	id (UUID), email (UK), password_hash, first_name, last_name, role (client/admin), company_name, created_at, is_active	1→N AuditRequest (submits / follows / receives)
AUDIT_PACK	id, reference, code (audit/security/protection/premium), name, description, duration_days, price	1→N AuditRequest (chosen_by)
AUDIT_REQUEST	id (UUID), client_id (FK), pack_id (FK), status (pending/in_progress/done/archived), scope_notes, submitted_at, updated_at, completed_at	N→1 User+Pack; 1→1 Carryover; 1→N Notification
AUDIT_REPORT	id (UUID), audit_request_id (FK UK), pdf_path, summary, findings (JSON), generated_at	1→1 AuditRequest
TRAINING_MODULE	id, title, content_md, duration_min, level, published_at	1→N Progress
TRAINING_PROGRESS	id, user_id, module_id, completed, started_at, completed_at	N→1 User + Module
NOTIFICATION	id, user_id, request_id, type, subject, status, sent_at	N→1 User + Request

Recommended indexes:

- USER.email UNIQUE
- AUDIT_REQUEST.status BTREE (frequent admin filtering)
- (AUDIT_REQUEST.client_id , AUDIT_REQUEST.created_at) composed (client dashboard)
- AUDIT_REPORT.audit_request_id SINGLE (1 report per request)

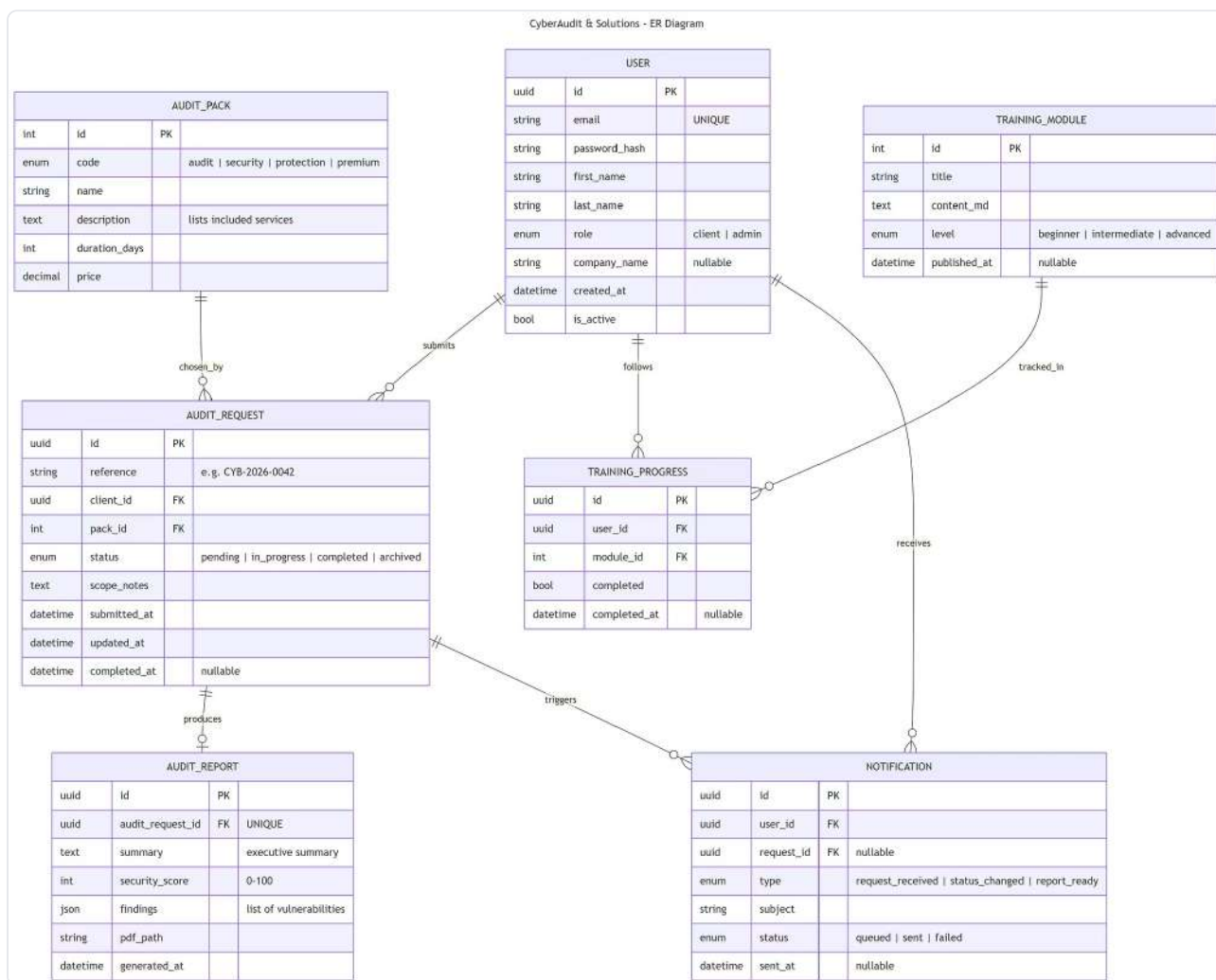


Figure 2 — Relational schema of the database

4.2 Back-end class diagram (Django models + services)

Class User

Represents a platform user, whether an SME client or administrator. Manages JWT authentication and role separation.

Attribute	Description
ID: UUID	Unique identifier (PK)
email: str	Email address (unique, indexed)
_password_hash: str	Hashed password (bcrypt / PBKDF2)
first_name, last_name	First name and last name
role: enum	client or admin
company_name: str	Company name (nullable)
is_active: bool	Account activated or deactivated
created_at: datetime	Account creation date

Methods: `check_password(raw)` , `has_role(role)`

AuditPack Class

Represents one of the 4 packages (Audit, Security, Protection, Premium). Reference data loaded via fixture.

<code>id: int</code>	unique identifier
<code>code: enum</code>	audit / security / protection / premium
<code>name: str</code>	Trade name
<code>description: str</code>	List of included services
<code>duration_days : int</code>	Estimated processing time
<code>Price: Decimal</code>	Package price

Class AuditRequest

Core business: the audit request submitted by an SME client. Cycle of 4 statuses (pending → in_progress → completed → archived).

<code>ID: UUID</code>	unique identifier
<code>reference: str</code>	Case number (e.g., CYB-2026-0042)
<code>client_id: FK → User</code>	SME client who submitted
<code>pack_id: FK → AuditPack</code>	Selected package
<code>status: enum</code>	pending / in_progress / completed / archived
<code>scope_notes: str</code>	Form details
<code>submitted_at, completed_at</code>	Cycle dates

Methods: `generate_reference()` , `update_status(new_status)` , `estimated_completion()`

Class Audit Report

Vulnerability report linked to a single AuditRequest, includes a visual score and a downloadable PDF.

<code>ID: UUID</code>	unique identifier
<code>audit_request_id: FK</code>	1-1 relationship
<code>summary: str</code>	Simplified Executive Summary
<code>security_score: int</code>	Score 0-100 (gauge)
<code>findings: JSON</code>	List of vulnerabilities
<code>pdf_path: str</code>	Path to the generated PDF
<code>generated_at</code>	Generation date

Methods: `build_pdf()` , `get_download_url()`

Class Notification

Traces all email communications sent via Celery.

ID: UUID	unique identifier
user_id: FK → User	RECIPIENT
request_id: FK	Request concerned (cancellable)
type: enum	request_received / status_changed / report_ready
status: enum	queued / sent / failed

Methods: send(), retry()

Classes TrainingModule & TrainingProgress

Cybersecurity awareness modules (phishing, MFA, GDPR...) with user progress tracking.

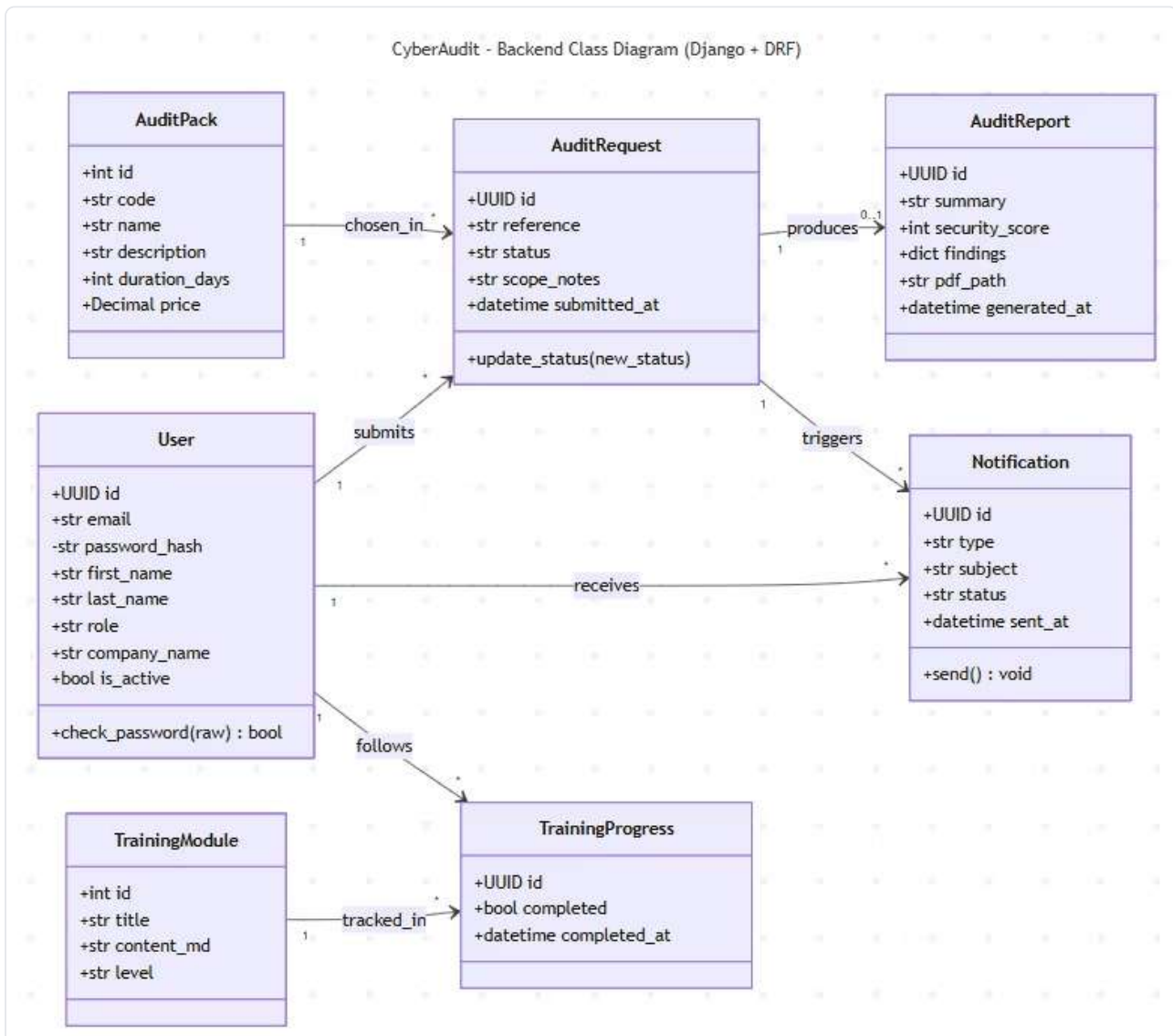


Figure 3 — Django back-end class diagram

Example - `accounts/models.py`

```
class User(AbstractBaseUser, PermissionsMixin):
    "Platform user — SME client or CyberAudit admin." ID
    = UUIDField(primary_key=True, default=uuid4) =
    e-mail      EmailField(unique=True, db_index=True)
    role        = CharField(choices=[('client','Client'),('admin','Admin')])
    company_name = CharField(max_length=200, blank=True)
    company_size = CharField(choices=[('TPE','TPE'),('PME','PME'),('ETI','ETI')]) sector
    = CharField(max_length=100, blank=True) =
    is_active    BooleanField(default=True)
    created_at    = DateTimeField(auto_now_add=True) =
    last_login    DateTimeField(null=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []

    def is_admin(self) -> bool: ... def
    is_client(self) -> bool: ...
```

Example - `audits/models.py`

```
class AuditPack(models.Model):
    """Catalog of the 4 audit packages offered.""" name
    = CharField(max_length=50) =
    description    TextField()
    duration_days  = PositiveIntegerField()
    price          = DecimalField(max_digits=8, decimal_places=2)

class AuditRequest(models.Model):
    Audit request submitted by a client. STATUS_CHOICES =
        [
            ('pending', 'En      waiting'),
            ('in_progress',      'In progress'),
            ('done', 'Finished'),
            ('archived',        'Archive'),
        ]
    id              = UUIDField(primary_key=True, default=uuid4)
    customer        = ForeignKey(User, on_delete=PROTECT, related_name='audit_requests') =
    pack            = ForeignKey(AuditPack, on_delete=PROTECT)
    status          = CharField(choices=STATUS_CHOICES, default='pending',
db_index=True)
    scope_notes     = TextField(blank=True)
    created_at      = DateTimeField(auto_now_add=True) =
    updated_at      DateTimeField(auto_now=True)
    completed_at    = DateTimeField(null=True, blank=True)

    def transition_to(self, new_status: str) -> None:
        """Changes the status, updates completed_at and triggers the notification.""" ...
```

```
def can_generate_report(self) -> bool:
    return self.status == 'done'
```

Business services (logic separate from models)

```
# audits/services.py
class AuditRequestService:
    @staticmethod
    def create_request(client: User, pack_id: int, scope: str) -> AuditRequest: ...

    @staticmethod
    def change_status(request: AuditRequest, new_status: str, by: User) -> None:
        """Change status + triggers Celery email.""" ...

# reports/services.py
class PDFReportGenerator:
    @staticmethod
    def generate(audit_request: AuditRequest) -> AuditReport:
        "Generates a PDF with WeasyPrint, stores the path, persists."

# notifications/tasks.py (Celery)
@shared_task
def send_status_change_email(audit_id: str, new_status: str) -> None: ...
```

4.3 Front-end components (React + Tailwind CSS)

The front-end is a React SPA communicating with the REST API via JWT. The hierarchy is structured into pages (complete views), reusable UI components, a global store, and API calling services.

Authentication & Routing

AuthStore—State global: `user` , `accessToken` , `refreshToken` Methods: `login` , `logout` , `refresh` , `hasRole` .

ProtectedRoute—Guard component, props: `allowedRoles` Redirects to `/login` Otherwise authenticated.

ApiClient—Wrapper axios which automatically adds the JWT to each request and manages the refresh token in case of 401.

Public pages

LoginPage—Login form. Submits to `AuthStore.login()` then redirects according to the role.

RegisterPage—SME customer registration form. Call `AuthStore.register()` .

SME Customer Area

Client Dashboard—Dashboard listing the requests of the logged-in customer.

AuditRequestForm—3-step form (pack, perimeter, confirmation). Displays 4 PackCards.

RequestDetail—Detailed view with timeline, PDF report, score. Components: `Status Badge` , `SecurityScoreGauge` , `FindingsList` .

CyberAudit Administrator Area

AdminDashboard—Paginated view of all requests with filters (status, package, date, client) and sorting.

Training module

TrainingPage—List of modules with progress indicator.

TrainingModuleViewer—Displays Markdown content. Security: cleanup via DOMPurify (anti-XSS, SMART #3).

Reusable UI components

Navbar, PackCard, Status Badge(grey/blue/green/purple), **SecurityScoreGauge**(circular gauge, red < 40, orange 40-70, green > 70), **RequestTable, Paging, ToastContainer**.

Main interaction flow — Request submission

1. The user logs in via LoginPage → AuthStore stores the JWT.
2. ProtectedRoute allows access to the ClientDashboard.
3. Click on "New request" → AuditRequestForm loads the packs.
4. Selecting a PackCard → AuditService.create() calls the Django API.
5. The back-end creates the AuditRequest and triggers the Celery email task.
6. The client is redirected to RequestDetail (StatusBadge "pending").
7. A notification is sent for each change of status on the admin side.

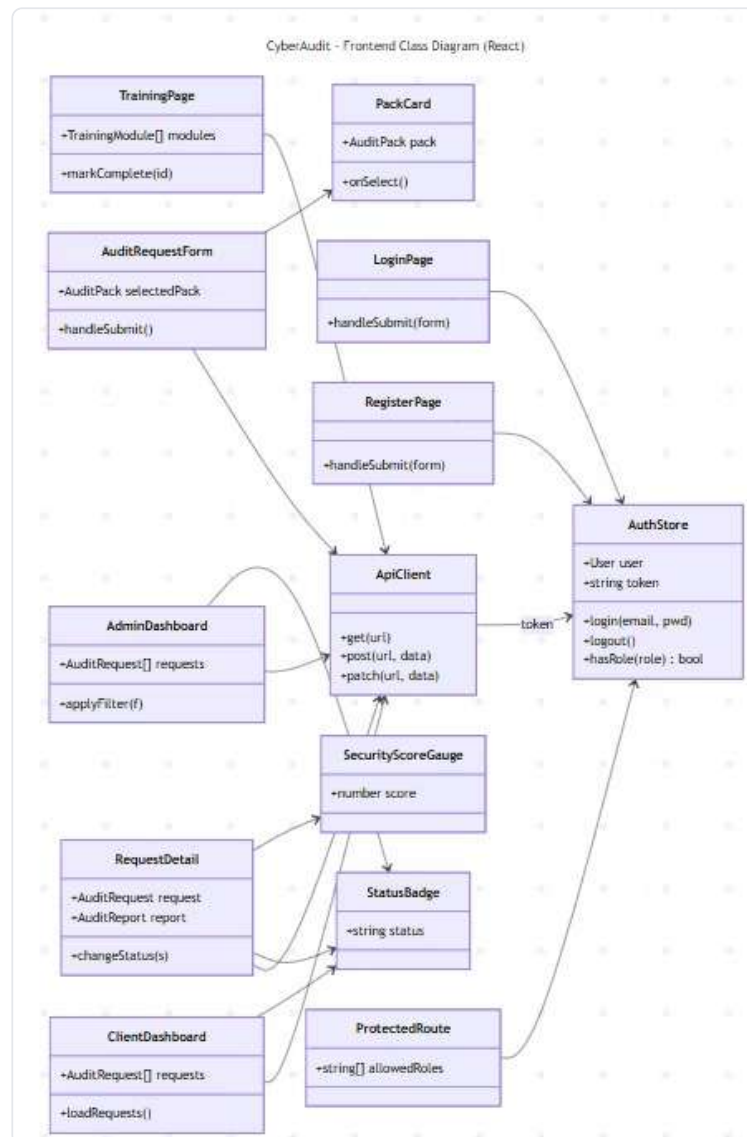


Figure 4 — React component tree

Component tree

src/	
├── components/	
│ ├── auth/	Login Form, RegisterForm, ProtectedRoute
│ ├── dashboard/	ClientDashboard, AdminDashboard, AuditRequestCard, StatusBadge
│ ├── audit/	AuditRequestForm, PackSelector, AuditDetailView
│ ├── training/	ModuleList, ModuleViewer
│ └── shared/	Header, Sidebar, Button, ErrorBoundary HomePage, LoginPage, RegisterPage, DashboardPage,
├── pages/	NewAuditPage, AuditDetailPage, TrainingPage useAuth,
├── hooks/	useAudits, useApi
├── services/	api.js (axios + JWT), auth/audits/reports services sanitize.js
├── utils/	(DOMPurify), validators.js
└── App.jsx	global routing

Conventions: Functional components + hooks (no classes), global state via Context API, sanitize() on all inputs.

4.4 Generation of the PDF vulnerability report

The PDF report is the SME client's primary deliverable: it consolidates the vulnerability analysis, security score, and recommended action plan. Its generation is entirely asynchronous to avoid blocking the admin interface.

technical stack

- **WeasyPrint**: Converts an HTML+CSS template into a print-quality PDF. Reuses the web's graphic design.
- **Celery**: Perform the task `generate_pdf_task` in the background.
- **Redis**: Celery queue (parallel scaling).
- **Storage**: `/media/reports/` to the MVP, S3 in V2; access via signed URL 15 min.

Structure of the generated PDF

Section	Content
Page 1 — Cover	CyberAudit logo, client name, case reference (CYB-2026-0042), date, confidentiality
Page 2 — Executive Summary	Overall score A to F (scale 0-100) + summary in 3 simplified lines
Page 3 — Audited Scope	List of assets analyzed (mail server, Wi-Fi router, workstations, etc.) + EBIOS RM methodology
Pages 4-N — Vulnerabilities	Table findings: severity (Critical/High/Medium/Low), active, description, recommendation
Final page — 30-day action plan	Weekly steps (S1, S2, S3, S4) with concrete actions and priorities
Annex	Glossary, references (ISO 27005, ANSSI), contact CyberAudit

Complete flow — from request to download

1. The admin clicks on "Generate PDF report" in AdminRequestDetail.
2. The front calls `POST /api/requests/{id}/generate-report` with the findings.
3. The `PdfReportService` Django creates the AuditReport record (summary, score, findings JSON).
4. He takes on the task `generate_pdf_task.delay(report_id)` and returns `202 Accepted`.
5. The worker Celery returns `report.html` with the data, then WeasyPrint produces the PDF.
6. File saved in `/media/reports/CYB-2026-0042.pdf`, `pdf_path` Updated.
7. Type notification `report_ready` Created, email sent via `send_email_task`.
8. On the client side, ClientDashboard displays "Download report" → `GET /api/reports/{id}/download` streaming.

PDF access security

- **JWT required** on `/api/reports/{id}/download` No anonymous access.
- **Permission** `IsAdminOrOwner` : 403 Forbidden otherwise.
- **Signed URL** with timestamp, expires after 15 minutes.
- **No browser cache**: `Cache-Control: no-store, private`.

- **Audit log**(V2): who, when, from which IP — GDPR compliant.

Generation failure handling

- **Automatic retry:** `autoretry_for=(Exception,)` , `max_retries=3` , exponential backoff (10s, 30s, 90s).
- **Intermediate status:** `pdf_path` empty while not generated → front displays "Generation in course...".
- **Failure notification:** after 3 failures, type `generation_failed` Sent to the admin.
- **Manual regeneration** via the "Generate PDF report" button.

4.5 Monitoring & observability

The application must remain traceable in production: we must know *Who did what, when, and why did it crash?* The strategy is based on 3 pillars: structured logs, error monitoring, and alerting.

4.5.1 Structured Logs (JSON)

Django configuration `structlog` to produce JSON logs consumable by Railway and a future ELK/Loki. Each log contains at least: `timestamp` , `level` , `request_id` (correlation), `user_id` , `endpoint` , `latency_ms` .

```
{
  "timestamp": "2026-05-18T09:24:11Z", "level":
  "INFO",
  "event": "audit_request.created",
  "request_id": "req-7f2a...", "user_id":
  "8f2a...c9",
  "audit_id": "CYB-2026-0042", "pack":
  "security",
  "latency_ms": 142
}
```

4.5.2 Error Monitoring — Sentry

- **Back-end:** `sentry-sdk[django,celery]` captures all Django and Celery exceptions with stack trace, user context, request payload (anonymized PII).
- **Front-end:** `@sentry/react` captures unhandled JS errors and promise rejections, plus replay sessions due to error.
- **Release tracking:** Each git tag pushes a Sentry release → followed by version regression.
- **Sample rate:** 100% of errors in MVP; 10% of transactions performance to stay within the free quota.

4.5.3 Business & Technical Metrics

Metric	Source	Alert threshold
API response time (p95)	Sentry Performance / Railway metrics	> 800 ms for 5 min
Error rate 5xx	Sentry	> 1% over 10 minutes
Consecutive Celery chess	Structured logs	≥ 3 on the same task

Metric	Source	Alert threshold
Active database connections	Railway Postgres metrics	> 80% of the pool
Disk space /media	Railway	> 85%
Audit requests per day	Django + admin dashboard	Growth metric, no alert

4.5.4 Alerting

- **Sentry** sends an email to the technical lead on any new unseen error, and a Slack DM (V2) if error rate > threshold.
- **Healthcheck**: endpoint `GET /api/health/` Checks database, Redis, SMTP. Monitored by UptimeRobot (free) every 5 min.
- **Celery failure notification**: after 3 retries, an admin email is sent (already documented in §4.4).

4.5.5 Application log audit (V2)

A table `AUDIT_LOG` Traces sensitive actions: connections, report downloads, Change of status, account deletion. Fields: `id, user_id, action, resource, ip, user_agent, created_at`. Storage for 1 year, in compliance with GDPR (see §7.5).

5. Sequence Diagrams

5.1 Registration + JWT Login

The user enters their email and password. The front end calls `POST /api/auth/register/`, Then `POST /api/auth/login/` Django validates credentials, generates access and refresh tokens, and returns All of this is done on the front end. The front end stores the tokens in AuthStore and redirects them to the dashboard corresponding to the role.

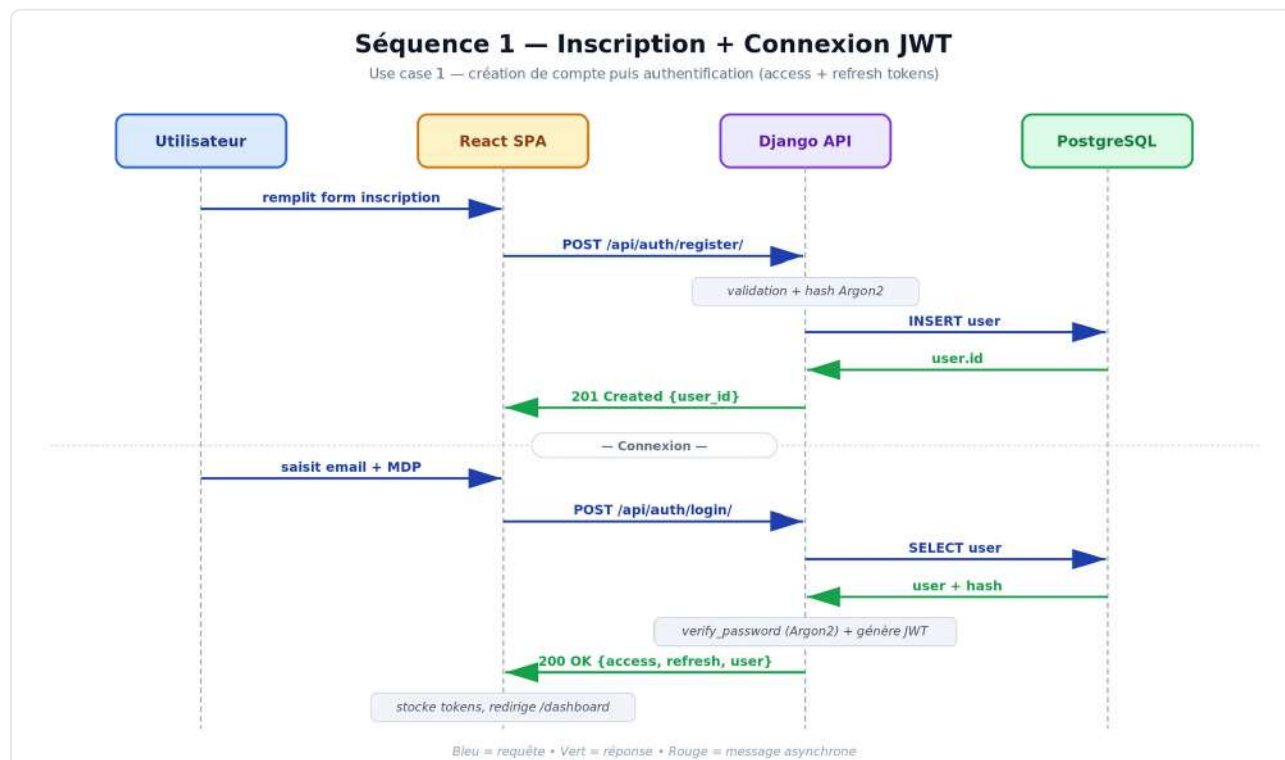


Figure 5 — JWT Registration + Login Sequence

5.2 Submitting an audit request (with async)

The client submits the audit form. The front office calls `POST /api/audits/` with the JWT in the header. Django validates via the DRF serializer, persists to the database, enrolls a Celery task to send an acknowledgment, and then responds `Created in 201` The Celery worker retrieves the task from Redis and sends the email. via SMTP.

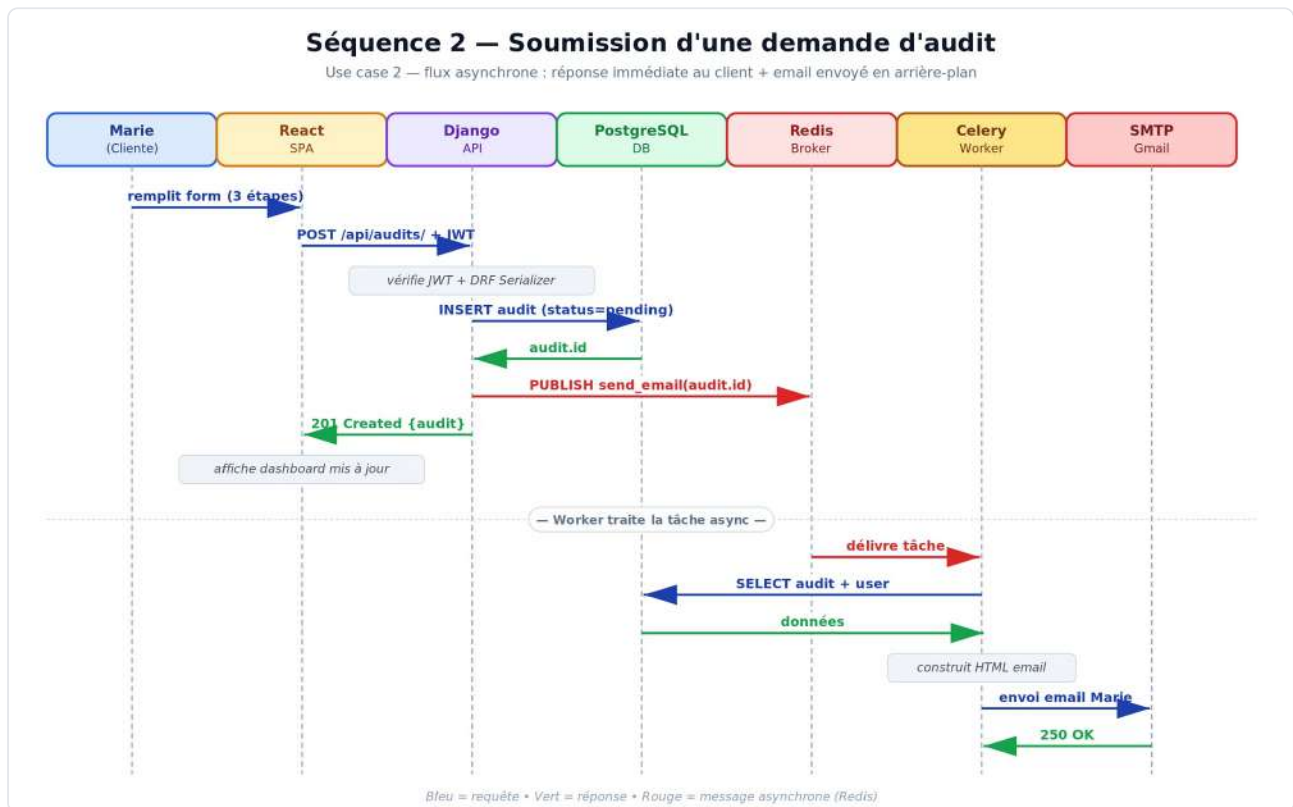


Figure 6 — Request Submission Sequence (Asynchronous Celery)

5.3 Generating the PDF report (admin)

The admin clicks on "Generate PDF report". The front end calls `POST /api/audits/{id}/generate-report/`. Django creates the `AuditReport` and threads the Celery task, returning `202 Accepted`. The worker Celery renders the HTML template, WeasyPrint produces the PDF, the file is stored, and an email notification is sent to the client.

Séquence 3 — Génération du rapport PDF (admin)

Use case 3 — l'admin déclenche, le worker génère, le client est notifié et récupère

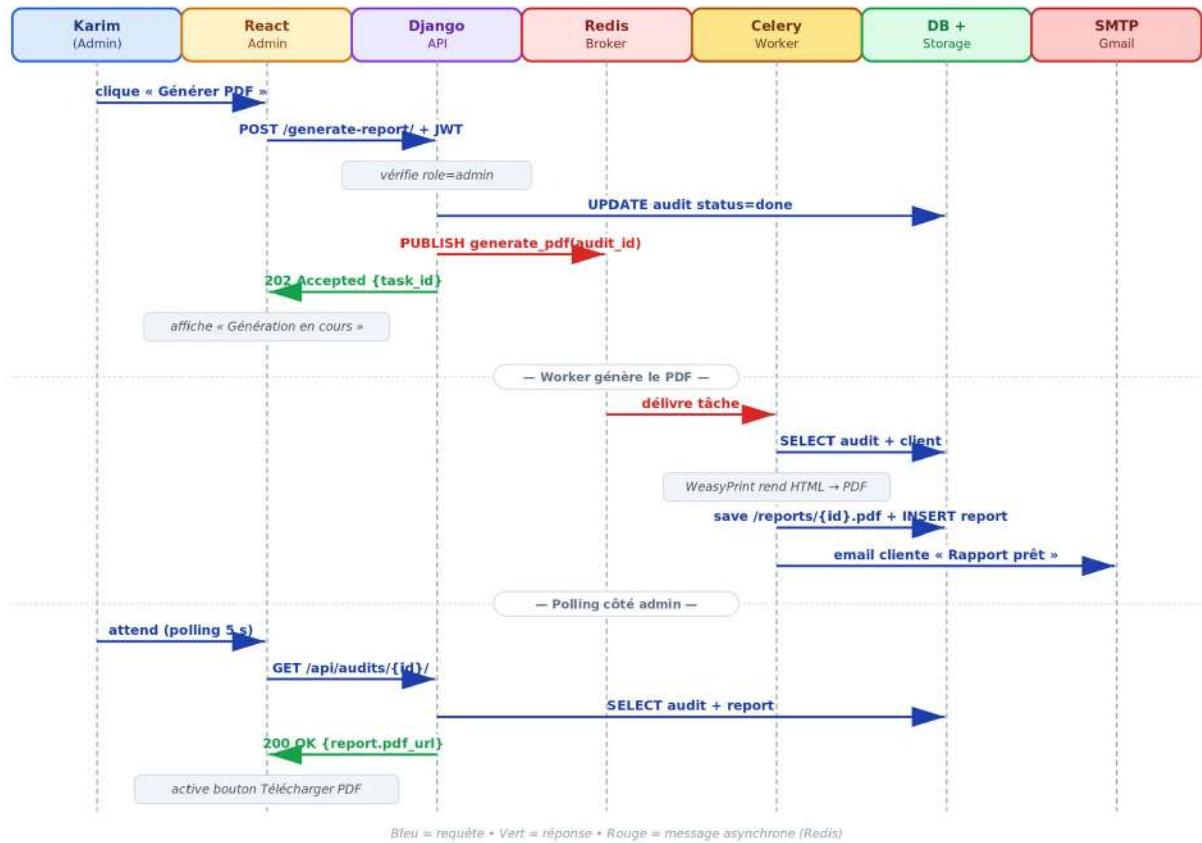


Figure 7 — PDF Report Generation Sequence

6. API Documentation

6.1 External APIs consumed

API external	Use	Justification
SMTP Gmail	Sending transactional emails (confirmation, status change, delivery report)	Free for up to 500 emails/day, simple setup, reliable. Migration possible to SendGrid/Mailgun V2.
Let's Encrypt	Automatic HTTPS certificates	Free, natively integrated into Vercel and Railway.

6.2 Internal API — REST Endpoints

Convention: prefix `/api/` JSON for requests and responses, JWT in `<token>` , standard HTTP codes.

Authorization: Bearer

6.2.1 Authentication — `/api/auth/`

Method	URL	Auth	Description	Body	Answer 2xx	Errors
POST	<code>/api/auth/register/</code>	—	Create a customer account	{email, password, company_name, company_size, sector}	201 {user, access, refresh}	400, 409 e-mail exists
POST	<code>/api/auth/login/</code>	—	Login + JWT tokens	{email, password}	200 {access, refresh, user}	401, 429 rate limit
POST	<code>/api/auth/refresh/</code>	—	Renews access	{refresh}	200 {access}	401 refresh expired
POST	<code>/api/auth/logout/</code>	✓	Invalid the refresh	{refresh}	204	401
POST	<code>/api/auth/password-reset/</code>	—	Send email reset	{e-mail}	200 (always, anti-leak)	—

6.2.2 Audit Requests — `/api/audits/`

Method	URL	Auth	Role	Description
GET	<code>/api/audits/</code>	✓	customer / admin	Paginated list (client → their requests / admin → all)
POST	<code>/api/audits/</code>	✓	customer	Create a new request

Method	URL	Auth	Role	Description
GET	/api/audits/{id}/	✓	owner / admin	Details of a request
PATCH	/api/audits/{id}/	✓	admin	Modifies status or notes
DELETE	/api/audits/{id}/	✓	admin	Archive (soft delete)
POST	/api/audits/{id}/ generatereport/	✓	admin	Launch PDF generation (Celery)
GET	/api/audits/{id}/report/	✓	owner / admin	Download the PDF (binary)

Example — Creating a request

```
POST /api/audits/
Authorization: Bearer eyJhbGc... Content-
Type: application/json

{ "pack_id": 2, "scope_notes": "Auditing the file server + 12 Windows workstations." }

HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "8f2a...c9",
  "client": "user-uuid",
  "pack": { "id": 2, "name": "standard", "status":
    "pending",
  "scope_notes": "Server audit...", "created_at":
    "2026-05-18T09:24:11Z"
}
```

6.2.3 Audit Packages — /api/packs/

Method	URL	Auth	Description
GET	/api/packs/	—	Public list of the 4 packs
GET	/api/packs/{id}/	—	Details of a pack

6.2.4 Training — /api/training/

Method	URL	Auth	Description
GET	/api/training/modules/	✓	List of available modules
GET	/api/training/modules/{id}/	✓	Module Contents
POST	/api/training/modules/{id}/start/	✓	Mark as started
POST	/api/training/modules/{id}/complete/	✓	Mark as finished

6.2.5 User Profile — /api/users/

Method	URL	Auth	Description
GET	/api/users/me/	✓	Current user profile
PATCH	/api/users/me/	✓	Edits his own profile
POST	/api/users/me/change-password/	✓	Change your password
DELETE	/api/users/me/	✓	Delete your account (GDPR)

6.3 Cross-cutting conventions

- **Paging:** ?page=1&page_size=20 , answer {count, next, previous, results}
- **Filters:** ?status=pending&ordering=-created_at
- **Errors:** unified format {error: "code", message: "Message FR", details: {...}}
- **Rate limiting:** 5 req/min on /auth/login/ And /auth/password-reset/ (IP-based)
- **Versioning:** No v1 in the MVP URL; in V2 if breaking changes → /api/v2/

7. SCM & QA Strategy

7.1 Source Code Management (Git)

Host: GitHub — repo then `Tommy-JOUHANS/cyberaudit-solutions` (private until the defense, public).

Branching strategy (lightweight Git flow)

hand ← Production only (protected, continuous deployment Vercel/Railway)
develop ← Integration branch (CI required)
feature/<name> ← New features (1 PR = 1 feature) fix/<name> ← Bug fixes

hotfix/<name> ← urgent fixes directly on main release/<v> ← stabilization before release (from W11)

Conventional Commits

feat(audits): added request form fix(auth): fixed expired JWT refresh
token docs(readme): updated tech stack test(audits): added CRUD
unit tests chore(deps): bumped Django 5.0.2 → 5.0.3
refactor(reports): extracted PDFGenerator in use

Pull Request Rules

Ruler	Detail
Mandatory review	Each PR is validated by the other member before merging.
CI green	Tests + lint OK before merge
No direct push	handAnddevelopprotected by GitHub Branch Protection
Structured description	"What's changing / Why / How to test"
Squash merge	1 PR = 1 commit indevelop ((own history)

7.2 QA Strategy

Pyramid of tests

 /\n /E2E\ 5% — Cypress (critical runs)
 /-----\
 / Integra. \ — API + DB integration tests (Django + DRF)
20% /-----\
 / Unitary \ — Jest (front) + pytest (back)
75% /-----\

Selected tools

Layer	Tool	Target
unit tests back	pytest + pytest-django	Models, services, serializers
API backtests	pytest + APIClient (DRF)	Auth endpoints, audits, reports
Front-end unit tests	Jest + React Testing Library	Components, hooks, utilities
E2E tests	Cypress	Registration → application → email (mock)
Safety tests	Postman (collection) + bandit	XSS, SQLi, brute-force, JWT tampering
UX testing	3 user sessions (W7-W10)	Persona Marie, journey < 3 min
Coverage	coverage.py + jest --coverage	Target: 80% minimum of critical apps
Lint	ruff + black; eslint + prettier	Pre-commit hook
Type checking	mypy; propTypes or TS (V2)	Starting with W4

7.3 Manual safety tests (W7-W10)

Test	Method	Expected result
Persistent XSS	<code><script>alert(1)</script></code> in scope_notes	Escaped / sanitized
SQL injection	' OR 1=1 --in email login	No leaks, ORM bind
Brute force	100 login attempts	Blocked from the 6th (rate-limit)
JWT tampering	To modifyrolein the payload	Invalid signature → 401
CSRF	POST without Authorization header	401
HSTS / CSP	HTTP headers in production	Present and strict

7.4 CI/CD Pipeline

GitHub Actions

```
#. github/workflows/ci.yml name:
CI
on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup-python 3.11
      - install: pip install -r backend/requirements-dev.txt
      - lint: ruff check. && black --check . pytest --cov=apps --
      - test: cov-fail-under=80
```

```

frontend:
  runs-on: ubuntu-latest
  steps:
    - checkout
    - setup-node 18
    - install: npm here
    - lint: npm run lint
    - test: npm test -- --coverage npm
    - build: run build

```

Continuous deployment

Environment	Trigger	Target	URL
Preview	All PR	Vercel preview + Railway branch deploy	pr-XX.preview.cyberaudit.app
Staging	Merge ondevelop	Vercel + Railway staging	staging.cyberaudit.app
Production	Merge onhand (release tag)	Vercel + Railway prod	cyberaudit.app

Release strategy

- **SemVer Versioning:** 0.1.0 (W4 partial MVP) → 0.5.0 (W6 async) → 1.0.0 (W12 defense)
- **Annotated git tag** with each release: `git tag -a v1.0.0 -m "MVP defense Holberton"`
- **Changelog** kept up to date in `CHANGELOG.md`

7.5 GDPR Compliance

The platform processes personal data (email, name, company) and sensitive data (vulnerability reports). Regulation (EU) 2016/679 (GDPR) imposes a strict framework that the MVP integrates from the design stage (*privacy by design*).

7.5.1 Processing Register

Treatment	Legal basis	Data	Shelf life
Creation of account	Execution contractual (art. 6.1.b)	email, name, surname, company, hashed password	Active account + 3 years after last login
Submission audit request	Execution contractual	scope, notes, chosen package	3 years from the date the file is closed (obligation an accountant)
Generation PDF report	Execution contractual	findings, score, recommendations	3 years, then automatic deletion
E- Notifications mail	Execution contractual	recipient, subject, sending status	1 year (debug)
Access logs & audit log	Legitimate interest (art. 6.1.f) — security	IP, user-agent, action, timestamp	1 year then automatic purging

Treatment	Legal basis	Data	Shelf life
No cookies essentials	Consent (art. 6.1.a)	None in MVP — no tracking	—

7.5.2 Rights of individuals

GDPR Law	Implementation
Access (art. 15)	GET /api/users/me/export>Returns a complete JSON of the account data. SLA: ≤ 30 days.
Correction (art. 16)	PATCH /api/users/me/allows direct modification by the user.
Deletion (art. 17 — "right to oblivion")	DELETE /api/users/me/triggers acascading deletion :User, AuditRequest, AuditReport, Notification, TrainingProgress. PDFs physically removed from storage. Logs anonymized (user_id replaced bydeleted-user)but preserved for legal integrity.
Portability (art. 20)	Export JSON via /api/users/me/export/in an open, reusable format.
Opposition (art. 21)	Disabling non-transactional emails via user preferences (V2).
Limitation (art. 18)	Fieldis_active=Falsestops access without deleting the account, upon request.

7.5.3 Technical and organizational measures

- **Resting encryption**:Postgres Railway encrypted AES-256. PDFs stored on encrypted volume.
- **Encryption in transit**:HTTPS required (Let's Encrypt), HSTS enabled with `max-age=31536000` .
- **Hashage**:passwords via `bcrypt` (Django by default, 12 rounds), never stored in plain text.
- **Minimization**:only the strictly necessary data is collected (no date of birth, no phone number not required).
- **Separation of access**:Only CyberAudit admins can see the reports; other clients are completely isolated (permission). `IsOwner`) .
- **Subcontractors**:Railway (hosting), Vercel (front), Gmail SMTP (sending emails) — all documented as subcontractors within the meaning of art. 28 in an internal register.

7.5.4 Cookies & consent banner

The MVP does not use **no third-party cookies or advertising trackers**Only a technical session cookie (JWT) is set, which is exempt from consent (CNIL recommendation "strictly necessary cookies"). The consent banner will be added in version 2 if Google Analytics or Plausible are introduced.

7.5.5 Data Breach Procedure

1. Detection via Sentry / monitoring / user reporting.
2. Immediate containment (token revocation, database isolation if necessary).
3. Risk assessment for people (severity, number of people affected).
4. Notification to the CNIL in the **72 hours**(art. 33) via [notifications.cnil.fr](https://www.cnil.fr/fr/notifications) .

5. Informing the users concerned if there is a high risk (art. 34).
6. Internal documentation of the incident in the violation log.

7.5.6 Legal Notice & Privacy Policy

Two public pages accessible from the footer: and [/legal/mentions](#) (publisher, host, contact) [/legal/privacy](#) (Privacy policy, including sections 7.5.1 to 7.5.5 in plain language). Link CNIL for complaint.

8. Appendices

8.1 Glossary

Term	Definition
MVP	Minimum Viable Product — minimum deliverable version
JWT	JSON Web Token — stateless authentication standard
MoSCoW	Prioritization method: Must / Should / Could / Won't
CRUD	Create / Read / Update / Delete
ORM	Object Relational Mapping — BDD abstraction
CI/CD	Continuous Integration / Continuous Deployment
GDPR	General Data Protection Regulation
HSTS	HTTP Strict Transport Security — anti-downgrade header
CSP	Content Security Policy — anti-XSS header
SemVer	Semantic Versioning — MAJOR.MINOR.PATCH
SPA	Single Page Application

8.2 Mapping Stage 3 ↔ RNCP Skills 5

RNCP reference 38038 — Web and Mobile Web Developer (TP issued by the Ministry of Labour).

RNCP block	Skill	Section
BC01 — Front-end	Designing an application	§2.3 (mockups)
BC01	Create static and dynamic interfaces	§4.3 (React components)
BC01	Develop the front-end	§3 (archi), §6 (API)
BC02 — Back-end	Create a database	§4.1 (ERD)
BC02	Develop the data access components	§4.2 (Django templates)
BC02	Develop the back-end	§6 (REST API)
BC02	Develop and implement components	§4.2 (services), §5 (sequences)
Transverse	Working in an agile team / Version control	§7 (SCM/QA)

8.3 Sources & technical references

Reference	Link
Django REST Framework — Documentation	https://www.django-rest-framework.org

Reference	Link
SimpleJWT — JWT for Django	https://django-rest-framework-simplejwt.readthedocs.io
Celery — Distributed Task Queue	https://docs.celeryq.dev
WeasyPrint — HTML to PDF	https://weasyprint.org
OWASP Top 10 — Security Reference Guide	https://owasp.org/www-project-top-ten
Conventional Commits	https://www.conventionalcommits.org
RNCP 38038	https://www.francecompetences.fr/recherche/rncp/38038
ANSSI — Cyber Threat Overview 2023	https://www.ssi.gouv.fr

8.4 Project Risk Matrix

Analysis of the technical and operational risks identified within the MVP scope. Each risk is assessed according to its probability (Low / Medium / High) and its impact (Low / Medium / High), then associated with a mitigation and a responsible party.

ID	Risk	Probability	Impact	Mitigation	Responsible
R-01	Worker failure Celery → PDF not generated	Average	Pupil	Automatic retry (3x, exponential backoff) + admin notification after final failure + manual "regenerate" button in AdminRequestDetail. Healthcheck UptimeRobot every 5 minutes.	James
R-02	Data leak personal (report Leaked PDF, database leak)	Weak	Pupil	URLs signed 15 min, JWT required, permission IsAdminOrOwner, audit log (§4.5), documented CNIL notification procedure within 72 hours (§7.5.5).	Tommy + James
R-03	Data loss Database (corruption, removal accidental)	Weak	Pupil	Daily automatic Railway backups (7-day retention on the free plan, 30 days in V2). Manual snapshot before each production migration. Restoration tests in Windows 10.	James
R-04	Force attack raw on /auth/ login/	Average	AVERAGE	Rate limiting 5 req/min per IP, temporary lockout after 5 failures, Sentry alert beyond 50 failures/h on the same IP. E2E tests covering the scenario (§7.3).	James
R-05	persistent XSS via scope_notesOr Markdown content training courses	Average	Pupil	DOMPurify front end, Django exhaust rear end, strict CSP (script-src 'self'). Manual security tests W7-W10 (§7.3).	Tommy

ID	Risk	Probability	Impact	Mitigation	Responsible
R-06	Gmail SMTP Quota reached (500 emails/day)	Average	AVERAGE	Daily counter monitoring. Migration to SendGrid (free up to 100/day, paid beyond that) or Mailgun as soon as volume exceeds 400/day. Celery backup queue to prevent email loss.	James
R-07	Delay in the W12 schedule (defense) Holberton)	Average	Pupil	Must-Have criteria fixed at W3, Should-Have criteria negotiable, Could-Have criteria excluding MVP. Points Tommy/James weekly. 1-week buffer planned in W11 for stabilization.	Tommy
R-08	Unavailability of a of the 2 developments (illness, exam)	Weak	AVERAGE	Pair programming in critical areas (auth, PDF generation). Up-to-date documentation enabling recovery. Trello/GitHub Project tasks assigned and prioritized.	Tommy + James
R-09	Major regression after deployment prod	Weak	Pupil	CI/CD pipeline required (lint + tests + 80% coverage), staging deployment before production, Railway/Vercel rollback in 1 click, Sentry monitoring release by release.	James
R-10	Non-compliance GDPR violation noted by the jury / CNIL	Weak	Pupil	Section §7.5 dedicated to legal notices and policy Online confidentiality from W6 onwards, right to be forgotten implemented before the defense, register of treatments up to date.	Tommy
R-11	Score Lighthouse Accessibility < 90	Average	AVERAGE	Lighthouse CI blocking in GitHub Actions, axe-core in unit tests, NVDA manual audit + keyboard in W8 (§3.4). Rapid iterations before the defense.	Tommy
R-12	Accommodation costs not mastered in V2	Weak	Weak	Free Railway and Vercel plans are sufficient for the MVP and presentation. Switch to a paid plan only upon commercial launch.	Tommy

8.4.1 Criticality Table

Cross-tabulation of probability × impact to visualize priorities:

	Low Impact	Medium Impact	High Impact
High Probability	—	—	—
Average Probability	—	R-04, R-06, R-11	R-01, R-05, R-07
Low Probability	R-12	R-08	R-02, R-03, R-09, R-10

THE 3 critical risks The priority targets (Average Probability × High Impact) are R-01 (Celery), R-05 (XSS) and R-07 (planning).

8.5 Internal project links

- Stage 1 — Project Charter (FR)
- Stage 2 — Planning & Gantt (FR)
- Figma Wireframes — to be published in W1 by Tommy
- MVP repository code — to be initialized in Windows 2020

Document prepared as part of Stage 3 — Technical Documentation, Holberton School Dijon, class of 2026.
Designed to be reused as is in the RNCP 5 certification file — Web and Mobile Web Developer.

Tommy Jouhans—Project Manager / Front-End Developer • **James ROUSSEL**—Technical expert /
Back-End Developer